

تقرير التدقيق المعماري وتحليل تأثير تحويل النظام إلى PWA

المشروع: بيت المندي - مطعم يماني (Bait Al Mandi)

التاريخ: 18 يونيو 2026

الإصدار: v0.1.0

المنصة: Next.js 14 + Supabase + PWA

المراجع: الملفات الفعلية للمشروع

فهرس المحتويات

1. [تحليل الهيكلية الحالية للنظام](#)
2. [جرد كامل لملفات PWA](#)
3. [شرح آلية عمل PWA الحالية](#)
4. [تحليل تأثير تحويل المشروع إلى PWA](#)
5. [تحليل التأثير على بنية النظام](#)
6. [تحليل المخاطر الحالية](#)
7. [توقع المشاكل المستقبلية](#)
8. [تحليل نقاط الضعف المعمارية](#)
9. [تقييم الجاهزية للإنتاج](#)
10. [التحسينات المطلوبة](#)
11. [الخلاصة النهائية](#)

المرحلة 1 : تحليل الهيكلية الحالية للنظام

1.1 الهيكلية العامة للمشروع

1.2 طبقات النظام (Layered Architecture)

النظام يتبع **Hybrid Architecture** تجمع بين Next.js App Router و Server Actions و Repository Pattern و Realtime Subscriptions و Offline-First PWA Architecture و

Next.js App Router Structure 1.3

```
src/app/ └─ layout.tsx # Root Layout: PwaProvider > OrderRealtimeProvider >
  SettingsProvider > ToastProvider └─ page.tsx # Homepage (Client Component) └─
    cart/page.tsx # Cart + Checkout └─ my-orders/page.tsx # My Orders + Leaflet Map +
      Checkout └─ track-order/[orderId]/ # Order Tracking page └─ offline/page.tsx # PWA
        Offline fallback page └─ t/[token]/page.tsx # Tracking by token └─ admin/ | └─
          layout.tsx # Admin layout (sidebar + auth) | └─ page.tsx # Admin dashboard | └─
            orders/page.tsx # Order management | └─ orders/actions.ts # Admin server actions |
              └─ menu/page.tsx # Menu management | └─ offers/page.tsx # Offers management | └─
                settings/page.tsx # Site settings | └─ gallery/page.tsx # Gallery management | └─
                  reviews/page.tsx # Reviews management | └─ reports/page.tsx # Reports dashboard |
                    └─ delivery/page.tsx # Delivery zones | └─ login/ # Admin login └─
                      gallery/page.tsx # Public gallery └─ contact/page.tsx # Contact page └─ api/ └─
                        auth/login/route.ts └─ resolve-zone/route.ts └─ reports/ # Reports API routes
```

Repository Pattern 1.4

النظام يستخدم **Dual Repository Pattern**:

- **Client-side repositories** (orderRepository.ts, menuRepository.ts, offerRepository.ts): تستخدم Supabase Browser Client مباشرة
- **Server-side repositories** (orderServerRepository.ts): تستخدم Supabase Server Client مع مصادقة cookies

ملاحظة: المستودعات لا تستخدم DTOs منفصلة، وتعيد nny أو الكائن الخام من Supabase.

1.5 قاعدة البيانات (PostgreSQL / Supabase)

21 جدولاً في قاعدة البيانات (حسب (src/db/schema.ts):

- users, admin_users, categories, items, item_prices
- offers, offer_items, cart_sessions, cart_items
- orders, order_items, order_offers, order_offer_items, order_sequences, order_status_history
- gallery_images
- reviews, site_settings, branches, audit_logs
- customer_tokens, push_subscriptions, scheduled_reports

17 ملف Migration في supabase/migrations (من 0000 إلى 0017).

1.6 دورة الطلب (Order Flow) - شرح تفصيلي

إنشاء الطلب:

1. المستخدم يستخدم يضيف أصنافاً إلى Zustand Store (مخزن في localStorage بمفتاح bam-cart-storage)
2. في صفحة الدفع (cart/page.tsx أو my-orders/page.tsx) يتم إنشاء idempotency_key فريد
3. تُستدعى Server Action createOrder () التي تعمل على الخادم:

◦ التحقق من صحة البيانات (اسم، هاتف، عنوان)

- جلب إعدادات التوصيل من site_settings
 - حساب المسافة عبر OSRM Route + Haversine Formula
 - حساب رسوم التوصيل (مع عوامل الطقس والذروة)
 - إنشاء رقم طلب عبر order_sequences مع optimistic locking و fallback
 - إدراج الطلب في orders مع idempotency_key
 - إدراج الأصناف في order_items
 - إدراج العروض في order_offers + order_offer_items
 - تسجيل order_status_history
 - إعادة رقم الطلب و tracking_token
4. العميل يُفتح WhatsApp أو يُوجّه إلى صفحة التتبع

تحديث الطلب:

1. المسؤول يضغط على زر تغيير الحالة في admin/orders/
2. Server Action updateOrderStatus():
 - تحقق صلاحية المسؤول
 - قراءة الـ version الحالي (Optimistic Concurrency Control)
 - تحديث مع 1 + version بشرط version = currentVersion
 - تسجيل في order_status_history و audit_logs
3. Realtime تُرسل فوراً لكل المتصفحات المتصلة
4. Cache Invalidation تُرسل رسالة INVALIDATE_CACHE إلى Service Worker

إدارة القوائم والعروض:

1. قائمة الطعام: menuRepository.ts تجلب من items + categories + item_prices
2. العروض: offerRepository.ts تجلب من offers + offer_items + menu_item join
3. التسعير: calculateOfferPrice() في lib/offer-pricing.ts

1.7 تدفق Realtime

هناك مساران منفصلان:

1. **OrderRealtimeProvider**: اشتراكات مباشرة في جدول orders (حسب ID أو token أو شامل)
2. **Cache Invalidation Channel**: اشتراك (event: *, schema: public) wildcard يُرسل رسائل INVALIDATE_CACHE إلى Service Worker لحذف الإدخالات المنتهية من Cache Storage

المرحلة 2 : جرد كامل لملفات PWA

2.1 جدول جرد ملفات PWA

الملف	الوظيفة	مستوى الخطورة	الاعتماديات
public/sw.js	Service Worker - إدارة الكاش، Background Sync، Push Notifications، Cache Invalidation	حاسم (Critical)	Cache Storage API، IndexedDB، fetch API، push API
public/manifest.json	Web App Manifest - تعريف التطبيق للتثبيت	عالي (High)	layout.tsx في metadata
src/lib/pwa/PwaProvider.tsx	React Context - إدارة حالة PWA (تثبيت، اتصال، تحديثات)	حاسم (Critical)	sw.js، indexeddb.ts، backgroundSync.ts، invalidation.ts
src/lib/pwa/types.ts	تعريفات الأنواع (SyncQueueItem، PwaState، CacheRule، CleanupConfig)	متوسط (Medium)	جميع ملفات PWA
src/lib/db/indexeddb.ts	IndexedDB Wrapper - تخزين محلي للطلبات والسلة وطابور المزامنة	حاسم (Critical)	types.ts
src/lib/sync/backgroundSync.ts	Background Sync - معالجة الطابور مع idempotency و cross-tab locking	حاسم (Critical)	indexeddb.ts، types.ts
src/lib/cache/invalidation.ts	Cache Invalidation - تحديث الكاش عبر Realtime	عالي (High)	types.ts، sw.js، postMessage
src/lib/push/pushNotifications.ts	Push Notifications - الاشتراك وإلغاء الاشتراك في الإشعارات	متوسط (Medium)	supabase.ts، VAPID keys
src/app/offline/page.tsx	Offline - تُعرض عند عدم الاتصال	منخفض (Low)	none
next.config.js	تكوين PWA (sw.js J headers و manifest.json)	متوسط (Medium)	next.js

2.2 تحليل الملفات

public/sw.js (Service Worker)

- **الموقع:** public/sw.js
- **الوظيفة:** يدير 5 أنواع من الكاش (static، pages، api-supabase، api-reports، assets)، يعالج Background Sync، يتعامل مع Push Notifications، وينفذ Cache Invalidation
- **لماذا موجود:** لتوفير دعم PWA (Offline، تثبيت، إشعارات)
- **ماذا يحدث إذا تعطل:** يفقد المستخدم القدرة على تثبيت التطبيق، تفشل الطلبات غير المتصلة، لا تعمل الإشعارات
- **تأثيره على النظام: حاسم** - تعطل Service Worker يعني تعطل PWA بالكامل

- **الموقع:** src/lib/pwa/PwaProvider.tsx
- **الوظيفة:** يسجل Service Worker، يدير حالة التثبيت، يراقب الاتصال بالإنترنت، ينظف IndexedDB دورياً
- **لماذا موجود:** لتوفير React Context للوصول إلى حالة PWA من أي مكون
- **ماذا يحدث إذا تعطل:** تتعطل واجهة المستخدم الخاصة بـ PWA، لا تظهر رسائل التحديث
- **تأثيره على النظام: حاسم** - يُغلف التطبيق بالكامل في layout.tsx (سطر 73)

src/lib/db/indexeddb.ts

- **الموقع:** src/lib/db/indexeddb.ts
- **الوظيفة:** واجهة IndexedDB مع 5 مخازن (orders, cart, syncQueue, tenantMeta, queueLocks)
- **لماذا موجود:** لتخزين البيانات محلياً للعمل دون إنترنت
- **ماذا يحدث إذا تعطل:** يفقد النظام القدرة على تخزين الطلبات محلياً، تتعطل Background Sync
- **تأثيره على النظام: حاسم** - أساس نظام Offline بالكامل

src/lib/sync/backgroundSync.ts

- **الموقع:** src/lib/sync/backgroundSync.ts
- **الوظيفة:** إدارة طابور المزامنة مع idempotency keys, cross-tab locks, broadcast channel، وثوابت exponential backoff
- **لماذا موجود:** لضمان عدم ازدواج الطلبات عند المزامنة
- **ماذا يحدث إذا تعطل:** قد تُرسل طلبات مكررة أو تُفقد طلبات
- **تأثيره على النظام: حاسم** - مسؤول عن سلامة الطلبات

src/lib/cache/invalidation.ts

- **الموقع:** src/lib/cache/invalidation.ts
- **الوظيفة:** اشتراك Realtime واحد (cache-invalidation-wildcard) يرسل رسائل INVALIDATE_CACHE لـ Service Worker
- **لماذا موجود:** لتحديث Cache Storage عند تغيير البيانات في Supabase
- **ماذا يحدث إذا تعطل:** يبقى الكاش قديماً، يرى المستخدم بيانات غير محدثة
- **تأثيره على النظام: عالي** - يؤدي لبيانات قديمة لكن لا يمنع النظام من العمل

المرحلة 3 : شرح آلية عمل PWA الحالية

3.1 آلية تثبيت التطبيق

مصدر التشغيل: PwaProvider.tsx (السطور 103-115)

آلية التثبيت:

1. المتصفح يُطلق حدث beforeinstallprompt (شروط: وجود HTTPS، manifest.json، تفاعل المستخدم)
2. PwaProvider يلتقط الحدث (يمنع السلوك الافتراضي) ويخزنه
3. حالة التثبيت تصبح installable
4. المكونات (مثل Navbar) يمكنها استدعاء installApp () لعرض زر التثبيت
5. installApp () تستدعي deferredPrompt.prompt () لتظهر نافذة التثبيت
6. بعد التثبيت، حدث ppinstalled يُغيّر الحالة إلى installed

شروط التثبيت:

- وجود manifest.json (موجود في public/manifest.json)
- اتصال HTTPS (إنتاج فقط)
- تفاعل المستخدم مع الصفحة (click/touch)
- المتصفح يدعم PWA

الملفات المعنية:

- public/manifest.json: display=standalone, orientation=portrait-primary
- manifest.json و sw.js خاصة بـ next.config.js: headers
- src/lib/pwa/PwaProvider.tsx: إدارة دورة التثبيت

3.2 آلية العمل دون إنترنت (Offline)

ما يعمل Offline:

- عرض الصفحات المخزنة مؤقتاً في PAGE_CACHE (استراتيجية StaleWhileRevalidate)
- عرض الأصول الثابتة من ASSET_CACHE (استراتيجية CacheFirst)
- عرض بيانات API من API_CACHE (استراتيجية NetworkFirst مع fallback للكاش. 5 maxAge دقائق)
- عرض صفحة /offline ك fallback للملاحة
- تخزين الطلبات الجديدة في IndexedDB syncQueue
- عرض ردود JSON وهمية لطلبات Supabase REST الفاشلة (سطر 305-308 في sw.js)

ما لا يعمل Offline:

- إنشاء الطلبات مباشرة (تُخزن في الطابور وترسل لاحقاً)
- تحديثات Realtime (تتطلب WebSocket)
- Push Notifications الجديدة
- البحث والتصفية المتقدمة
- الخريطة (Leaflet) - تتطلب تحميل tiles

كيف يتم التعامل مع الطلبات أثناء انقطاع الإنترنت:

1. المستخدم يضغط على "تأكيد الطلب"
2. Server Action تفشل (لا يوجد اتصال)
3. Service Worker يلتقط الفشل في fetch event (سطر 476-521 في sw.js)
4. يُخزّن الطلب في IndexedDB syncQueue مع idempotency key وحالة pending
5. يُسجّل حدث Background Sync
6. المستخدم يتلقى رد { queued: true } مع HTTP 202
7. عند استعادة الاتصال، Background Sync تُشغّل processSyncQueue()
8. الطلب يُعاد إرساله مع idempotency key لمنع الازدواج

3.3 آلية Background Sync

المسار الكامل:

```
↓ (cart/page.tsx) المستخدم ↓ يضغط على "تأكيد الطلب" (بدون إنترنت) واجهة التطبيق
↓ (سطر 476 sw.js) Service Worker (Network Error) تفشل Server Action createOrder()
↓ syncQueue تخزين العنصر في IndexedDB (indexeddb.ts) catch block ← تفشل fetch()
↓ sync حدث (سطر 594 sw.js) Background Sync (syncQueue store) Queue (pending: حالة)
↓ مع Supabase REST API (سطر 330 processSyncQueue) Service Worker (pwa-sync-queue)
إدراج الطلب مع منع الازدواج عبر idempotency key Database (PostgreSQL)
idempotency_key
```

شرح المراحل:

المرحلة 1: الالتقاط (Interception)

- Service Worker يعترض طلب fetch الفاشل
- يُنشئ idempotency_key إذا لم يكن موجوداً
- يُحقن idempotency_key في body للجداول التي تتطلب ذلك (orders)
- يُخزّن العنصر في IndexedDB مع idempotencyKey, metadata (url, method, headers, body, idempotencyKey, retryCount)

المرحلة 2: التخزين المؤقت (IndexedDB)

- المخزن: syncQueue
- المفتاح: id فريد (sync-{timestamp}-{random})
- الفهارس: status_idx, idempotency_idx
- الحالة: pending
- الحد الأقصى لعدد المحاولات: 3

المرحلة 3: المزامنة (Background Sync)

- يتم تسجيل حدث pwa-sync-queue عبر self.registration.sync.register()
- عندما يكون الجهاز متصلاً، المتصفح يُشغّل حدث sync
- Service Worker يستدعي processSyncQueue()

المرحلة 4: معالجة الطابور (processSyncQueue)

- قفل cross-tab عبر IndexedDB (queueLocks store)
- استرداد العناصر المعلقة + العناصر العالقة في processing (أكثر من 30 ثانية)
- لكل عنصر:
 - التحقق من maxRetries (3 افتراضياً)
 - تحديث الحالة إلى processing
 - حقن idempotency_key في header و body
 - إرسال fetch
 - إذا نجح: تحديث الحالة إلى completed
 - إذا كان 409 (duplicate): اعتباره completed (idempotency)
 - إذا كان 4xx: تحديث إلى failed
 - إذا كان 5xx أو Network Error: backoff أسّي (1s, 2s, 4s, ... 30s كحد أقصى) وإعادة المحاولة
- تحرير القفل

المرحلة 5: الإنهاء

- Broadcast Channel يُعلم التبويبات الأخرى بتحرير القفل
- يمكن للتبويبات الأخرى محاولة المعالجة

3.4 آلية Cache**إنشاء الكاش:**

- أثناء install event: يتم فتح STATIC_CACHE وتحميل (/logo.png, /offline, /) PRECACHE_URLS (/manifest.json)
- أثناء fetch: إنشاء الكاش تلقائياً عند استراتيجيات CacheFirst و NetworkFirst و StaleWhileRevalidate

أنواع الكاش:

اسم الكاش	الاستراتيجية	الاستخدام	TTL
static-v1	CacheFirst	الأصول الثابتة (JS, CSS, fonts)	دائم
pages-v1	StaleWhileRevalidate	الصفحات (HTML navigation)	دائم (validates on each request)
api-supabase-v1	NetworkFirst	استعلامات Supabase REST	5 دقائق
api-reports-v1	NetworkFirst	استعلامات API Routes	بدون TTL
assets-v1	CacheFirst	الصور والملفات الثابتة	دائم

تحديث الكاش:

- Static Cache: يُعاد أثناء install (عند تغيير CACHE_VERSION)

- Page Cache: يُحدّث في الخلفية (StaleWhileRevalidate)
- API Cache: يُحدّث من الشبكة أولاً (NetworkFirst)، مع fallback للكاش عند الفشل
- Cache Invalidation: عبر رسائل INVALIDATE_CACHE من التطبيق

حذف الكاش:

- أثناء activate event: حذف جميع الكاشات غير المطابقة للإصدار الحالي
- أثناء Cache Invalidation: حذف إدخالات محددة حسب الجدول (orders, items, categories, إلخ)
- عند تغيير CACHE_VERSION: حذف جميع الإصدارات القديمة

:Cache Invalidation

- جدول TABLE_TO_CACHE في invalidation.ts يحدد أي الكاشات يجب مسحها لكل جدول
- Service Worker يستخدم INV_PATTERNS لتحديد URL patterns لكل جدول
- الرسالة INVALIDATE_CACHE ترسل عبر postMessage إلى Service Worker
- Service Worker يحذف الإدخالات المطابقة ثم يُعلم جميع التبويبات بـ CACHE_INVALIDATED

التعامل مع البيانات القديمة:

- API Cache: NetworkFirst تعطي الأولوية للشبكة، الكاش هو fallback فقط
- StaleWhileRevalidate: تعرض الكاش فوراً ثم تُحدّث في الخلفية
- Settings Cache: TTL 60 ثانية في الذاكرة
- Reports Cache: TTL 120 ثانية في الذاكرة

3.5 آلية Realtime

إنشاء القنوات:

- OrderRealtimeProvider يُنشئ قنوات منفصلة لكل اشتراك (حسب order ID, token, شامل، أو جدول)
- Cache Invalidation Channel واحد wildcard لجميع جداول public

الاشتراكات:

- subscribeToOrder(orderId, handler): filter id=eq.{orderId}
- subscribeToToken(token, handler): filter tracking_token=eq.{token}
- subscribeToAllOrders(handler): بدون filter (كل orders)
- subscribeToTable(tableName, handler, filter): أي جدول مع filter اختياري

إعادة الاتصال:

- Supabase JS SDK يعيد الاتصال تلقائياً
- OrderRealtimeProvider لا يعالج إعادة الاتصال بشكل صريح (يعتمد على SDK)

التفاعل مع الكاش:

- عند استلام حدث Realtime (خاص بـ Cache Invalidation)، يتم إرسال INVALIDATE_CACHE إلى Service Worker
- Service Worker يحذف الإدخالات المطابقة من الكاش
- يتم تحديث refreshSignal في PwaProvider لإعادة تحميل المكونات
- المكونات التي تشترك في حدث cache-invalidated يمكنها إعادة جلب البيانات

المرحلة 4 : تحليل تأثير تحويل المشروع إلى PWA

4.1 مقارنة قبل وبعد PWA

الجانب	قبل PWA	بعد PWA	هل تغيرت طريقة العمل؟
إنشاء الطلب	Server Action مباشرة. إذا فشل، يرى المستخدم خطأ.	Server Action مع fallback: إذا فشل، يُخزّن في IndexedDB ويُرسل لاحقاً. المستخدم يرى { queued: true }.	نعم - ظهرت طبقة تخزين مؤقتة (IndexedDB) وطبقة مزامنة (Background Sync)
متابعة الطلب	Refresh يدوي أو Realtime مباشر	Realtime + تحديث الكاش تلقائياً عند تغيير البيانات	نعم جزئي - تحسّن الأداء مع دعم Offline
لوحة الإدارة	اتصال مباشر بـ Supabase	اتصال مباشر + استفادة من Service Worker Cache لتحسين السرعة	لا - نفس الآلية مع تحسين أداء بسيط
تحديثات الوقت الحقيقي	اشتراكات Supabase فقط Realtime	اشتراكات + Cache Invalidation Channel إضافي	نعم - ظهر مسار جديد للتحديثات (Realtime → SW → UI Refresh Cache Delete)
الأداء	يعتمد على سرعة الشبكة	CacheFirst للملفات الثابتة، StaleWhileRevalidate للصفحات، NetworkFirst للـ API	تحسّن - الصفحات تظهر من الكاش فوراً
قاعدة البيانات	كل الطلبات مباشرة على Supabase	بعض الطلبات تُخزّن محلياً أولاً ثم تُزامن	نعم - ظهرت طبقة تخزين وسيطة (IndexedDB) قاعدة البيانات
الشبكة	كل شيء يتطلب اتصالاً	بعض الوظائف تعمل Offline وتُزامن لاحقاً	تغير جذري - النظام أصبح Offline-Capable
التخزين المحلي	فقط localStorage (Zustand cart)	localStorage + Cache Storage API + IndexedDB (3 أنظمة تخزين منفصلة)	تغير جذري - 3 طبقات تخزين محلي بدلاً من طبقة واحدة

4.2 تحليل التغييرات

هل تغيرت طريقة العمل؟

نعم، تغيرت بشكل جذري. أصبح النظام **Offline-First** حيث يمكن للمستخدم إنشاء طلب دون اتصال بالإنترنت. هذا يتطلب:

1. طبقة تخزين محلية موثوقة (IndexedDB)
2. طبقة مزامنة ذكية (Background Sync مع idempotency)
3. طبقة كاش (Cache Storage API مع استراتيجيات متعددة)
4. طبقة إبطال كاش (Realtime → Cache Invalidation)

هل تغير منطق النظام؟

نعم، لكن **بشكل محدود**. منطق الأعمال الأساسي (حساب التوصيل، إنشاء رقم الطلب، التحقق من Minimum Order) بقي في Server Actions. التغيير الأساسي هو **إضافة طبقة وسيطة** بين المستخدم وقاعدة البيانات.

هل تغيرت دورة الطلب؟

نعم، أصبح هناك مساران:

1. **المسار المباشر:** مستخدم → Server Action → Database (عند الاتصال)
2. **المسار المؤجل:** مستخدم → IndexedDB → Background Sync → Database (بدون اتصال)

هل ظهرت طبقات إضافية؟

نعم، 5 طبقات إضافية:

1. IndexedDB Layer (التخزين المحلي)
2. Sync Queue Layer (طابور المزامنة)
3. Cache Storage Layer (الكاش)
4. Cache Invalidation Layer (إبطال الكاش)
5. Service Worker Layer (اعتراض الطلبات وإدارة الكاش)

هل ظهرت مخاطر جديدة؟

نعم، مخاطر كبيرة:

1. **ازدواج الطلبات** (رغم idempotency، لا يزال هناك احتمالية في حالات edge)
2. **فقدان المزامنة** (تعطل IndexedDB، خطأ في Service Worker)
3. **تضارب البيانات** (مستخدم يعدّل طلباً محلياً وآخر عبر النظام)
4. **مشاكل الكاش** (Cache Poisoning, Stale Cache)
5. **Race Conditions** (بين عدة تبويبات)
6. **استهلاك التخزين** (IndexedDB + Cache Storage قد يصل للحد المسموح)

المرحلة 5 : تحليل التأثير على بنية النظام

المكون	هل تأثر؟	ماذا تغير؟	لماذا؟	التقييم
Architecture	نعم	إضافة طبقة Service PWA (Worker, IndexedDB, Background Sync, Cache)	لتوفير دعم Offline وتجربة تطبيق سطح المكتب	إيجابي - تحسين تجربة المستخدم بشكل كبير
Repository Pattern	لا	بقي بدون تغيير. المستودعات لا تزال تتصل بـ Supabase مباشرة	PWA يعمل في طبقة منفصلة فوق المستودعات	محايد - لم يتأثر، لكن يمكن تحسينه بإضافة Repository Offline
Business Logic	لا	منطق الأعمال بقي في Server Actions (createOrder, updateOrderStatus)	PWA يعترض الشبكة قبل أن تصل إلى Business Logic	إيجابي - الفصل بين PWA و Business Logic يحافظ على الأمان
قاعدة البيانات	نعم جزئي	إضافة idempotency_key إلى orders (Migration 0016, 0017)	لمنع إدراج طلبات مكررة من Background Sync	إيجابي - حماية ضد الازدواج
Realtime	نعم	إضافة Cache Invalidation Channel منفصل	لتحديث Cache Storage عند تغيير قاعدة البيانات والمصدر	إيجابي - يضمن اتساق البيانات بين الكاش والمصدر
Security	لا	بقي Supabase RLS + Admin Middleware + Server-only secrets	PWA Service Worker لا يغير نموذج الأمان (يعمل بنفس anon key)	محايد - لكن Service Worker يمكنه اعتراض جميع الطلبات (مخاطرة أمنية)
Maintainability	نعم	8 ملفات PWA جديدة + منطق مزامنة معقد	التعقيد زاد بشكل ملحوظ	سليبي - زيادة التعقيد المعماري والصيانة
Scalability	نعم	Background Sync يُقلل الضغط على الخادم في أوقات الذروة (تجميع الطلبات)	بدلاً من طلب فوري، يُخزّن محلياً ويُرسَل لاحقاً	إيجابي - تحسين scalability عبر تقليل الطلبات المترابطة

5.1 تحليل مفصل

التأثير على Architecture:

النظام انتقل من **Client-Server Model** إلى **Offline-First Model**. هذا تغيير جذري في طريقة التفكير:

- قبل PWA: واجهة → خادم → قاعدة بيانات
- بعد PWA: واجهة → (كاش / IndexedDB) → خادم → قاعدة بيانات

التأثير على Base de données:

التغيير الوحيد في قاعدة البيانات هو إضافة idempotency_key إلى جدول orders مع unique partial index. هذا ضروري لمنع الازدواج لكنه يضيف تعقيداً في التعامل مع الأخطاء (معالجة Conflict 409).

التأثير على Security:

مخاطرة أمنية محتملة: Service Worker يمكنه اعتراض جميع طلبات HTTP وتعديلها. رغم أن الشيفرة الحالية لا تفعل ذلك بشكل صار، إلا أن أي خطأ في Service Worker يمكن أن يسرّب بيانات أو يعدّل طلبات.

- ملف: public/sw.js سطر 454-467 (stripSensitiveHeaders) - يحاول إزالة ال headers الحساسة. لكنه لا يمنع تخزين sensitive data في IndexedDB.

التأثير على Maintainability:

التعقيد زاد بشكل ملحوظ:

- 8 ملفات PWA جديدة
- منطق مزامنة معقد (locks, heartbeats, backoff, retries)
- 3 أنظمة تخزين محلي (localStorage, Cache Storage, IndexedDB)
- مساران لكل عملية (Online vs Offline)

المرحلة 6 : تحليل المخاطر الحالية

6.1 مخاطر حرجية (Critical)

#	الخطر	الوصف	السبب	الملف	السيناريو	التأثير	الاحتمالية
CR-01	Duplicate Orders	إنشاء طلب مكرر رغم idempotency	Service Worker يرسل الطلب قبل وصول استجابة الخادم، أو يفشل idempotency check	sw.js:369-377, orders.ts:468-478	مستخدم يضغط على "تأكيد" مرتين سريعاً في وضع Offline → طلبان يخزانان في IndexedDB → عند الاتصال يُرسلان → idempotency key قد لا يمنع الازدواج إذا لم يصل الخادم بعد	خسارة مالية، استياء العميل	متوسطة
CR-02	Queue Corruption	فساد طابور المزامنة	خطأ في IndexedDB، تعطل المتصفح، انقطاع الكهرباء	indexeddb.ts, backgroundSync.ts	متصفح يتعطل أثناء كتابة syncQueue → بيانات غير كاملة → فقدان الطلب	فقدان طلبات العملاء	منخفضة
CR-03	Lost Updates	فقدان تحديثات الحالة	تحديثان متزامنان من	orderServerRepository.ts:73-98, orders.ts:743-791	مسؤولان يحاولان تحديث حالة	إزعاج للمسؤولين.	متوسطة

#	الخطر	الوصف	السبب	الملف	السيناريو	التأثير	الاحتمالية
			مسؤولين (OCC يفشل)		الطلب نفسه في نفس الوقت → أحدهما يفشل برسالة تم تعديل الطلب من شخص آخر	لكنه محمي بـ OCC	
CR-04	Realtime Disconnection غير معالج	فقدان التحديثات الفورية	عدم معالجة إعادة اتصال Realtime بعد قطع WebSocket	OrderRealtimeProvider.tsx	اتصال الإنترنت يقطع ويعود → الاشتراكات لا تعاد بشكل صحيح → المسؤول لا يرى الطلبات الجديدة	تأخير في معالجة الطلبات	عالية

6.2 مخاطر عالية (High)

#	الخطر	الوصف	السبب	الملف	التأثير
HI-01	Service Worker Stale	Service Worker قديم لا يخدم الصفحات الجديدة	PwaProvider يسجل SW لكن لا يعالج skipWaiting بشكل تلقائي	PwaProvider.tsx:52-83	المستخدم يرى إصداراً قديماً من التطبيق
HI-02	Cache Poisoning	تخزين ردود خاطئة في الكاش	NetworkFirst يخزن أي رد 200 في الكاش، حتى لو كان خطأ	sw.js:286-288	عرض بيانات خاطئة للمستخدمين غير المتصلين
HI-03	Stale Cache	عرض بيانات قديمة بسبب TTL طويل	API Cache TTL 5 دقائق طويل جداً للطلبات	sw.js:526	عرض حالة طلب قديمة لمدة 5 دقائق
HI-04	No Sync Retry Limit Reset	إعادة محاولة غير محدودة بعد updateStatus	تعديل الحالة إلى pending لا يعيد retryCount	backgroundSync.ts:247-257	عنصر قد يفشل بعد 3 محاولات ولا يُحاول مجدداً
HI-05	Cross-Tab Race Condition	تضارب بين تبويين في معالجة الطابور	queuelocks تعتمد على IndexedDB الذي قد يكون بطيئاً	indexeddb.ts:304-332	تبويان يعالجان نفس العنصر

6.3 مخاطر متوسطة (Medium)

#	الخطر	الوصف	الملف
ME-01	IndexedDB Quota	وصول تخزين IndexedDB للحد المسموح	indexeddb.ts:103-111 (معالجة جزئية فقط)
ME-02	Multi-Tab Cart Sync	سلة التسوق لا تتزامن بين التبويبات	cartStore.ts (مع Zustand مع localStorage فقط)
ME-03	No Order Rollback on Partial Failure	فشل جزئي في إنشاء الطلب (items OK, offers FAIL)	orders.ts:538-589
ME-04	VAPID Key غير مهياً	VAPID_PUBLIC_KEY فارغ → الإشعارات لا تعمل	pushNotifications.ts:4-5
ME-05	BroadcastChannel غير متوفر	بعض المتصفحات لا تدعم BroadcastChannel	backgroundSync.ts:156-160, 229-240

6.4 مخاطر منخفضة (Low)

#	الخطر	الوصف	الملف
LO-01	Offline Page بسيطة جداً	لا تقدم خيارات للمستخدم	offline/page.tsx
LO-02	SW لا يعالج FormData	يفترض أن كل ال body هو JSON	sw.js:374-375
LO-03	لا يوجد Periodic Sync	لا تزامن دوري في الخلفية	sw.js (لا يوجد periodicSync)

المرحلة 7 : توقع المشاكل المستقبلية (محاكاة الأحمال)

جدول نقاط الانهيار المتوقعة

المكون	100 مستخدم	500 مستخدم	1000 مستخدم	5000 مستخدم	نقطة الانهيار	مستوى الخطورة
Supabase DB	100% طبيعي	95% طبيعي (قد تظهر تأخيرات في الاستعلامات المعقدة)	80% حاجة لمعظم الاستعلامات (تحت 200ms)	40% حاجة ماسة connection pooling + (PgBouncer)	تحميل DB مع +1000 طلب متزامن ل orders	عالي
Supabase Realtime	100% طبيعي	100% طبيعي (حد Supabase 500 اتصال متزامن)	80% (قد نفترق من الحد)	50% (تجاوز حد 500 اتصال)	500 اتصال متزامن هو حد Supabase Realtime	عالي
Background Sync	100% طبيعي	95% (قد تظهر تأخيرات في معالجة الطابور)	70% (قفل واحد للطابور قد يسبب عنق زجاجة)	30% (فشل locks, طابور طويل جداً)	Single Queue Lock لكل التطبيق	حرج

المكون	100 مستخدم	500 مستخدم	1000 مستخدم	5000 مستخدم	نقطة الانهيار	مستوى الخطورة
IndexedDB	100% طبيعي	95% طبيعي	85% (قد تظهر مشاكل المساحة)	50% (حد 5-10MB للتطبيق في بعض المتصفحات)	حد التخزين (5-50MB حسب المتصفح)	متوسط
Cache Storage	100% طبيعي	100% طبيعي	90% طبيعي	60% (حد 50-100MB للتطبيق)	حد التخزين (غير محدد لكن محدود)	متوسط
Service Worker	100% طبيعي	100% طبيعي	90% طبيعي	70% (قد تظهر مشاكل في معالجة الأحداث المتزامنة)	معالجة fetch events لعدد كبير من التيوبيات	منخفض
Supabase Limits	آمن	آمن	قريب من الحد	تجاوز الحد	500 اتصال Realtime. 50000 طلب/ساعة (خطة Free)	حرج

:Pro Subscription Recommended

عند تجاوز 500 مستخدم متزامن، ستحتاج إلى:

- خطة Supabase Pro (اتصالات Realtime غير محدودة)
- PgBouncer لـ connection pooling
- قراءة متوسعة (Read Replicas) لتقليل ضغط DB
- Rate Limiting لـ Background Sync لمنع تجاوز Supabase Rate Limits

المرحلة 8 : تحليل نقاط الضعف المعمارية

Architectural Debt 8.1

نقطة الضعف	الموقع	وصف المشكلة	التأثير المستقبلي
Inconsistent Repository Usage	/repositories	بعض الصفحات تستخدم repositories، بعضها يستخدم supabase مباشرة	صعوبة تغيير مصدر البيانات، صعوبة الاختبار
Type Safety Gap	/repositories	معظم الدوال ترجع iny بدلاً من أنواع محددة	أخطاء وقت التشغيل (Runtime Errors) غير مكتشفة في وقت الترجمة

نقطة الضعف	الموقع	وصف المشكلة	التأثير المستقبلي
Dual Cart Systems	cartStore.ts + indexeddb.ts	سلة في localStorage وأخرى في IndexedDB دون تزامن	ارتباك في البيانات، فقدان عناصر السلة
No Offline Repository	-	لا يوجد طبقة Repository تدعم Offline تلقائياً	كل مكون يحتاج لمعالجة Offline يدوياً
Realtime Reconnection Gap	OrderRealtimeProvider.tsx	لا معالجة صريحة لإعادة اتصال Realtime	فقدان التحديثات بعد انقطاع الشبكة

Scalability Issues 8.2

المشكلة	الوصف	التأثير
Single Queue Lock	processSyncQueue تستخدم قفلاً واحداً لجميع المستأجرين (tenants)	عنق زجاجة عند 1000+ مستخدم
No Pagination for syncQueue	getPending () تجلب جميع العناصر المعلقة بدون ترقيم	استهلاك ذاكرة عالي مع طابور طويل (آلاف العناصر)
No Batch Processing	كل عنصر في الطابور يُعالج بشكل فردي	أداء ضعيف مع طابور كبير
No Rate Limiting	لا يوجد معدل لطلبات Supabase	تجاوز حدود Supabase API (50000 طلب/ساعة)
Realtime Channel per Order	OrderRealtimeProvider ينشئ قناة لكل اشتراك فردي	آلاف القنوات عند 5000 مستخدم (حد ~500 Supabase اتصال)

Technical Debt 8.3

العنصر	الموقع	المشكلة
console.log left in production	orders.ts, page.tsx, backgroundSync.ts	دوال console.log عديدة لل debugging
Any Types	جميع المستودعات	استخدام any بدلاً من TypeScript strict types
Hardcoded Values	sw.js سطر 13-18	PRECACHE_URLS في الكود بدلاً من التكوين
Duplicate Code: Checkout	my-orders/page.tsx و cart/page.tsx	منطق الدفع مكرر في صفحتين منفصلتين
Error Handling Gap	backgroundSync.ts سطر 208-213	catch block يسمح الخطأ الأصلي

Single Points Of Failure 8.4

نقطة الفشل	الموقع	التأثير
Service Worker	public/sw.js	أي خطأ في SW يُعطل PWA بالكامل
PwaProvider	layout.tsx:73	تعطل PwaProvider يُعطل التطبيق بالكامل (يُغلق كل شيء)
Supabase Project	-	توقف Supabase يُعطل كل شيء (لا يوجد fallback)

نقطة الفشل	الموقع	التأثير
Queue Lock	queueLocks	تعطل القفل قد يؤدي لمعالجة مزدوجة
Idempotency Key	orders.ts:457-459	أي خطأ في idempotency يؤدي لازدواج الطلبات

Performance Bottlenecks 8.5

عنق الزجاجة	الموقع	الوصف
createOrder Server Action	orders.ts:171-621	دالة واحدة 450 سطرأً تفعل +10 استعلامات DB و Route API
Order Number Generation	orders.ts:10-71	10 محاولات مع optimistic locking (قد يفشل عند الضغط العالي)
getOrdersClient	orderRepository.ts	يجلب جميع العناصر من order_items بدون Pagination داخل الـ join
processSyncQueue	backgroundSync.ts	معالجة تسلسلية (sequential) لكل عنصر في الطابور
Cache Invalidation	invalidation.ts	حذف إدخالات الكاش واحداً تلو الآخر في حلقة

المرحلة 9 : تقييم الجاهزية للإنتاج

جدول التقييم (من 100)

المجال	التقييم	%	التعليق
Architecture	72/100	72%	جيد لكن يعاني من نقص في Type Safety و Modularity
Database	80/100	80%	Schema جيد مع indexes و OCC. لكن يفتقر لبعض القيود (constraints)
PWA Layer	75/100	75%	تنفيذ جيد مع idempotency و locks. لكن معالجة الأخطاء غير مكتملة
Offline Support	70/100	70%	أساسي (لا يمكن عرض القائمة أو إدارة السلة Offline بشكل كامل)
Realtime	65/100	65%	جيد لكن لا يعالج إعادة الاتصال وفقدان القنوات
Security	85/100	85%	جيد جداً (RLS, Middleware, Server Actions)
Performance	70/100	70%	createOrder دالة ثقيلة. لا يوجد batch processing
Reliability	68/100	68%	يعتمد بشكل كبير على Supabase. لا يوجد fallback كافي
Maintainability	60/100	60%	نقص في التوثيق. types any. كود مكرر
Scalability	55/100	55%	Single queue lock. لا rate limiting. حد 500 اتصال Realtime

النتيجة النهائية للمشروع

المجموع الكلي: 700/1000 = 70%

التقييم: المشروع في حالة جيدة لكنه يحتاج تحسينات قبل الإطلاق للإنتاج الفعلي (Production-Ready) مع تحفظات).

المرحلة 10: التحسينات المطلوبة

10.1 عاجل جداً (قبل الإطلاق) – Priority: Critical

#	الوصف	السبب	التأثير	درجة الصعوبة
F01	معالجة إعادة اتصال Realtime	إضافة onReconnect handler في OrderRealtimeProvider لإعادة الاشتراك بعد انقطاع WebSocket	يمنع فقدان التحديثات الفورية بعد انقطاع الشبكة	متوسطة
F02	إزالة console.log من الإنتاج	إزالة جميع دوال console.log من orders.ts و backgroundSync.ts و page.tsx	يحسّن الأداء، يمنع تسريب معلومات حساسة	سهلة
F03	VAPID Key Check	التحقق من وجود VAPID_PUBLIC_KEY قبل محاولة الاشتراك في الإشعارات	يمنع أخطاء JavaScript في وحدة التحكم	سهلة
F04	تحديث clearCart بعد المزامنة	مسح السلة من IndexedDB بعد إتمام المزامنة بنجاح	يمنع إعادة إرسال طلبات قديمة	متوسطة

10.2 قصير المدى – Priority: High

#	الوصف	السبب	التأثير	درجة الصعوبة
F05	إضافة 严格 Types للمستودعات	استبدال ny بأنواع TypeScript محددة	يكشف أخطاء وقت الترجمة، يحسّن الصيانة	متوسطة
F06	دمج صفحتي الدفع	دمج my-orders/page.tsx و cart/page.tsx في مكون واحد	يقلل تكرار الكود، يسّهل الصيانة	متوسطة
F07	Periodic Sync	إضافة sync دوري (كل 30 دقيقة) للمتصفحات التي تدعمه	تحسين موثوقية المزامنة	متوسطة
F08	Rate Limiting للـ Background Sync	إضافة throttle لعدد طلبات Supabase في الثانية	يمنع تجاوز حدود Supabase API	متوسطة
F09	إضافة Offline Capability للقائمة	تخزين القائمة (items, categories, offers) في IndexedDB للعرض Offline	تحسين تجربة المستخدم غير المتصل	صعبة

10.3 متوسط المدى – Priority: Medium

#	الوصف	السبب	التأثير	درجة الصعوبة
F10	Batch Processing للطابور	معالجة عناصر الطابور في batches بدلاً من عنصر واحد	تحسين أداء Background Sync بشكل كبير	صعبة
F11	إضافة Offline Repository Layer	طبقة Repository تعيد البيانات من IndexedDB تلقائياً عند عدم الاتصال	تبسيط معالجة Offline في جميع المكونات	صعبة
F12	فصل createOrder إلى خدمات أصغر	تقسيم دالة createOrder (450 سطر) إلى خدمات متخصصة	تحسين الاختبار والصيانة وإعادة الاستخدام	صعبة
F13	إضافة اختبارات لـ PWA	اختبارات لـ Background Sync و IndexedDB و Service Worker	يضمن استقرار PWA بعد التعديلات	متوسطة
F14	إضافة Error Boundary لـ PWA	React Error Boundary حول PwaProvider	يمنع تعطل التطبيق بالكامل عند فشل PWA	سهلة

10.4 طويل المدى – Priority: Low

#	الوصف	السبب	التأثير	درجة الصعوبة
F15	إضافة IndexedDB Quota Monitoring	مراقبة استخدام التخزين وتحذير المستخدم قبل الامتلاء	يمنع فقدان البيانات بسبب تجاوز السعة	متوسطة
F16	إضافة Multi-Tab Cart Sync	مزامنة سلة التسوق بين التبويبات عبر BroadcastChannel	يحسن تجربة المستخدم مع عدة تبويبات	متوسطة
F17	تحسين Offline Page	إضافة خيارات مفيدة (عرض القائمة، رقم الهاتف) في صفحة Offline	تجربة أفضل للمستخدم غير المتصل	سهلة
F18	إضافة Distributed Queue (Upstash Redis)	استبدال IndexedDB lock بـ Redis lock لتحسين scalability	يسمح بمعالجة آلاف المستخدمين دون عنق زجاجة	صعبة

الخلاصة النهائية

⚠ جاهز للإنتاج بشروط

الأدلة:

نقاط القوة (Positive):

1. **Idempotency** مطبق بشكل جيد: idempotency_key في orders مع unique partial index (م0016، 0017 igrations)، حقن idempotency في body و header، معالجة 409 Conflict
2. **Optimistic Concurrency Control** عبر version في orders يمنع التحديثات المتضاربة
3. **Security** جيد: Supabase Middleware، RLS policies، فصل admin auth

4. **Repository Pattern** ✓ لفصل منطق الوصول للبيانات
5. **PWA Structure** ✓ متكاملة: Service Worker, Manifest, IndexedDB, Background Sync, Push Notifications, Cache Invalidation
6. **Cross-tab Locking** ✓ عبر IndexedDB queueLocks يمنع معالجة مزدوجة للطاير
7. **Exponential Backoff** ✓ مع random jitter في Background Sync
8. **TTL-based Cache** ✓ للـ API Cache (5 دقائق) و Settings Cache (60 ثانية)
9. **Offline Fallback** ✓ صفحة offline/ واستجابة JSON للـ API

نقاط الضعف (Negative) التي تمنع الإطلاق المباشر:

1. **Realtime Reconnection** ✗ غير معالج: فقدان التحديثات بعد انقطاع WebSocket
2. **console.log** ✗ في الإنتاج: page.tsx و orders.ts تحتوي على عشرات الـ console.log التي تسرّب بيانات العملاء
3. **No Rate Limiting** ✗ Background Sync: يمكن أن يتجاوز حدود Supabase API (50000 طلب/ساعة)
4. **Single Queue Lock** ✗ عنق زجاجة للمستقبل
5. **Type Safety Gap** ✗ استخدام any في معظم المستودعات
6. **كود مكرر** ✗ صفحتا دفع منفصلتان (my-orders و cart) بـ 1200 سطر إجمالاً
7. **No Backup for Supabase Failure** ✗ النظام يعتمد كلياً على Supabase (database, auth, realtime, storage)

الشروط المطلوبة للإطلاق:

1. إزالة console.log من الإنتاج
2. إضافة معالجة إعادة اتصال Realtime
3. التحقق من VAPID keys
4. اختبار تحميل أساسي (100 مستخدم متزامن)
5. مراجعة أمنية أخيرة (خاصة بـ Service Worker و IndexedDB)

متى سيكون جاهزاً تماماً للإطلاق:

بعد تنفيذ التحسينات F01-F04 (عاجل) في غضون 1-2 أيام عمل.

نهاية التقرير

إعداد: تدقيق معماري آلي - Principal Software Architecture Review

تاريخ التدقيق: 18 يونيو 2026